

**POSTS AND TELECOMMUNICATIONS INSTITUTE
OF TECHNOLOGY
FACULTY OF INFORMATION TECHNOLOGY 1**



**SOFTWARE ARCHITECTURE
AND DESIGN
AI-service02**

Lecturer: Trần Đình Quế
Member: B22DCDT059 - Trần Anh Dũng
Class: E22CNPM01

Hà Nội, 2026

MỤC LỤC

1. Mô tả AISERVICE.....	3
1.1. Mục tiêu của AI Service.....	3
1.2. Vị trí AI Service trong kiến trúc hệ thống.....	3
1.3. Thành phần chức năng chính của AI Service.....	3
1.4. Luồng xử lý tổng quát.....	3
1.5. Endpoint AI tiêu biểu sử dụng trong hệ thống.....	4
1.6. Giá trị mang lại khi tích hợp vào e-commerce.....	4
2. DATA.....	4
2.1 20 dòng data.....	5
2.2 Giải thích 8 hành vi (behaviors).....	5
2.3 Ý nghĩa các cột dữ liệu.....	5
3. Xây dựng 3 mô hình RNN, LSTM, biLSTM.....	6
3.1. Mục tiêu.....	6
3.2. Dữ liệu và tiền xử lý.....	6
3.3. Code triển khai 3 mô hình.....	6
3.4. Kết quả đánh giá thực nghiệm.....	11
3.5. Lý giải vì sao chọn LSTM.....	12
4. Xây dựng KnowledgeBase Graph với Neo4j.....	12
4.1. Mục tiêu.....	12
4.2. Dữ liệu đầu vào và quy mô graph.....	13
4.3. Ý nghĩa của KB_Graph.....	13
4.4. Quy mô graph.....	13
4.4. 20 dòng mẫu dữ liệu graph.....	14
4.5. Ảnh chi tiết quan hệ người dùng và đồng mua.....	15
4.6. Kết luận.....	15
5. Tính năng RAG Chat và Tích hợp E-commerce.....	16
5.1. Mục tiêu.....	16
5.2. Câu 2c: RAG và Chat dựa trên KB_Graph.....	16
5.3. Minh chứng câu 2c.....	16
5.4. Câu 2d: Tích hợp vào hệ e-commerce.....	17
5.5. Kết luận.....	19

1. Mô tả AISERVICE

1.1. Mục tiêu của AI Service

AI Service được thiết kế như một lớp trí tuệ trung tâm cho hệ e-commerce y tế, phục vụ đồng thời 3 nhiệm vụ:

1. Tư vấn hội thoại y tế theo ngữ cảnh người dùng (chat).
2. Truy xuất tri thức từ Knowledge Base và Knowledge Graph để tăng độ tin cậy câu trả lời (RAG + Graph).
3. Gợi ý sản phẩm cá nhân hóa dựa trên hành vi và ngữ cảnh sử dụng.

Trong hệ thống hiện tại, AI Service là một service hợp nhất, giúp giảm phân mảnh logic AI và đơn giản hóa tích hợp từ API Gateway.

1.2. Vị trí AI Service trong kiến trúc hệ thống

AI Service chạy độc lập và giao tiếp với các thành phần sau:

1. API Gateway: nhận request từ UI web và chuyển tiếp tới AI endpoint.
2. Pharmacy Service: lấy dữ liệu sản phẩm thực tế để hiển thị/đề xuất.
3. Neo4j: lưu và truy vấn quan hệ tri thức dạng đồ thị.
4. FAISS: lưu chỉ mục vector để truy xuất ngữ nghĩa.
5. Dataset hành vi người dùng: làm đầu vào cho pipeline học máy và recommendation.

Điểm quan trọng: dữ liệu sản phẩm chuẩn được lưu ở Pharmacy Service; AI Service chỉ tham chiếu để suy luận và gợi ý.

1.3. Thành phần chức năng chính của AI Service

Dựa trên hiện trạng code:

1. API Chat: xử lý câu hỏi người dùng, phân tích intent và tạo phản hồi theo ngữ cảnh tri thức.
2. API Knowledge Base: theo dõi trạng thái KB/FAISS, rebuild chỉ mục khi cần.
3. API Graph Query: truy vấn tri thức từ Neo4j theo triệu chứng/chủ đề.
4. API Recommendation: sinh danh sách sản phẩm gợi ý cho từng user.
5. API Health: giám sát trạng thái hoạt động của service.

Tham chiếu triển khai:

- ai-service/main.py
- api-gateway/app/views.py
- api-gateway/app/views.py
- api-gateway/api_gateway/urls.py

1.4. Luồng xử lý tổng quát

Luồng xử lý AI trong hệ thống được mô tả như sau:

1. Người dùng thao tác trên giao diện web (chat, tìm kiếm, xem gợi ý).
2. API Gateway nhận request và chuyển tới AI Service.
3. AI Service lấy ngữ cảnh tri thức:
 - Từ Neo4j (quan hệ triệu chứng-bệnh-sản phẩm).
 - Từ FAISS (tài liệu/đoạn tri thức liên quan).
4. AI Service kết hợp logic intent + ngữ cảnh để tạo câu trả lời hoặc danh sách gợi ý.
5. Gateway render kết quả về UI e-commerce.

1.5. Endpoint AI tiêu biểu sử dụng trong hệ thống

Các endpoint cốt lõi:

1. Chat: POST /api/chat
2. KB stats: GET /api/kb/stats
3. KB build: POST /api/kb/build
4. KB sources: GET /api/kb/sources
5. Graph query: GET /api/graph/query
6. Recommend: GET /api/recommend/
7. Health: GET /health/

Các endpoint này đã được đóng gói trong service thống nhất, thuận lợi cho vận hành và demo.

1.6. Giá trị mang lại khi tích hợp vào e-commerce

AI Service giúp hệ thống đạt 3 giá trị chính:

1. Tăng chất lượng tư vấn: câu trả lời có căn cứ tri thức thay vì phản hồi mẫu.
2. Tăng khả năng cá nhân hóa: recommendation dựa trên user_id và hành vi.
3. Tăng trải nghiệm mua sắm: gợi ý và chat xuất hiện trực tiếp trong giao diện web, không tách rời luồng mua hàng.

=> AI Service trong dự án không chỉ là chatbot đơn lẻ, mà là một tầng AI hợp nhất gồm Chat + KB/Graph + Recommendation, tích hợp trực tiếp vào kiến trúc microservices e-commerce. Cách tổ chức này đáp ứng đúng mục tiêu đề bài cho các phần 2b, 2c, 2d và tạo nền cho phần đánh giá mô hình ở 2a.

2. DATA

Dữ liệu sử dụng cho bài toán được lấy từ file ai-service/data_user500.csv.

Bộ dữ liệu gồm 500 người dùng và 8 hành vi tương tác chính trong hệ e-commerce y tế.

2.1 20 dòng data

User_Product_Actions									
user_id	product_id	action	timestamp	category	#	price	rating	session_duration	
U264	P095	view	2026-01-01 09:45:38	stomach_nutrition	290221			56	
U264	P095	click	2026-01-01 10:01:38	stomach_nutrition	298431			1068	
U409	P095	view	2026-01-01 10:50:13	stomach_nutrition	328994			1439	
U409	P095	wishlist	2026-01-01 11:08:13	stomach_nutrition	288616			196	
U264	P075	add_to_cart	2026-01-01 11:14:38	stomach_nutrition	315030			880	
U409	P095	view	2026-01-01 12:22:13	stomach_nutrition	305694			711	
U264	P075	purchase	2026-01-01 12:28:38	stomach_nutrition	380401	5.0		1240	
U264	P075	review	2026-01-01 12:48:38	stomach_nutrition	312654	3.0		1142	
U264	P075	view	2026-01-01 12:55:38	stomach_nutrition	343475			167	
U370	P004	view	2026-01-01 13:34:37	herbal_extract	218031			1349	
U409	P095	share	2026-01-01 13:39:13	stomach_nutrition	310628			666	
U370	P084	compare	2026-01-01 14:02:37	herbal_extract	177386			930	
U264	P075	click	2026-01-01 14:15:38	stomach_nutrition	374701			451	
U409	P095	wishlist	2026-01-01 14:19:13	stomach_nutrition	292415			207	
U409	P095	share	2026-01-01 14:27:13	stomach_nutrition	289966			1459	
U370	P084	add_to_cart	2026-01-01 14:57:37	herbal_extract	187084			1259	
U370	P054	purchase	2026-01-01 15:06:37	herbal_extract	364865			798	
U264	P075	compare	2026-01-01 15:22:38	stomach_nutrition	379623			1126	
U409	P095	wishlist	2026-01-01 15:26:13	stomach_nutrition	327937			1059	
U370	P054	purchase	2026-01-01 15:29:37	herbal_extract	310369	3.0		542	

2.2 Giải thích 8 hành vi (behaviors)

- view: người dùng xem sản phẩm.
- click: người dùng bấm vào chi tiết sản phẩm.
- add_to_cart: người dùng thêm sản phẩm vào giỏ hàng.
- wishlist: người dùng thêm sản phẩm vào danh sách yêu thích.
- compare: người dùng so sánh sản phẩm với sản phẩm khác.
- share: người dùng chia sẻ sản phẩm.
- review: người dùng đánh giá/bình luận sản phẩm.
- purchase: người dùng hoàn tất mua sản phẩm.

2.3 Ý nghĩa các cột dữ liệu

- user_id: mã người dùng.
- product_id: mã sản phẩm.
- action: hành vi tương tác.
- timestamp: thời điểm phát sinh hành vi.
- category: nhóm sản phẩm.
- price: giá tại thời điểm tương tác.
- rating: điểm đánh giá (chỉ có khi có review hoặc purchase kèm rating).
- session_duration: thời gian phiên tương tác.

3. Xây dựng 3 mô hình RNN, LSTM, biLSTM

3.1. Mục tiêu

Ở câu 2a, em xây dựng 3 mô hình học chuỗi hành vi người dùng gồm RNN, LSTM và BiLSTM để dự đoán hành vi tiếp theo, sau đó so sánh bằng các độ đo phù hợp và chọn ra `model_best`.

3.2. Dữ liệu và tiền xử lý

Dữ liệu đầu vào là `data_user500.csv`, gồm 8 hành vi:

`view`, `click`, `add_to_cart`, `purchase`, `wishlist`, `review`, `share`, `compare`.

Pipeline tiền xử lý:

1. Mã hóa action thành `action_idx`.
2. Mã hóa category thành `category_idx`.
3. Trích xuất đặc trưng thời gian theo chu kỳ (sin/cos giờ và thứ).
4. Chuẩn hóa `price`, `session_duration`, `rating`.
5. Tạo chuỗi theo user để dự đoán action kế tiếp.

Thông số đã dùng:

1. `seq_len = 8`
2. `n_features = 25`
3. `n_actions = 8`
4. `learning_rate = 0.0008`

3.3. Code triển khai 3 mô hình

3.3.1. Code xây dựng 3 mô hình (RNN, LSTM, BiLSTM)

```
def build_rnn_model(seq_len: int, n_features: int, n_actions: int):
    _require_tensorflow()
    from tensorflow.keras.layers import Dense, Dropout, Input, LSTM, SimpleRNN
    from tensorflow.keras.models import Sequential

    model = Sequential(
        [
            Input(shape=(seq_len, n_features)),
            LSTM(64, return_sequences=True),
            Dropout(0.2),
            SimpleRNN(32),
            Dropout(0.2),
            Dense(16, activation="relu"),
            Dense(n_actions, activation="softmax"),
        ]
    )
    model.compile(optimizer="adam", loss="categorical_crossentropy", metrics=["accuracy"])
    return model
```

```
def build_lstm_model(seq_len: int, n_features: int, n_actions: int):
    _require_tensorflow()
    from tensorflow.keras.layers import Dense, Dropout, Input, LSTM
    from tensorflow.keras.models import Sequential

    model = Sequential(
        [
            Input(shape=(seq_len, n_features)),
            LSTM(128, return_sequences=True),
            Dropout(0.3),
            LSTM(64),
            Dropout(0.3),
            Dense(32, activation="relu"),
            Dense(n_actions, activation="softmax"),
        ]
    )
    model.compile(optimizer="adam", loss="categorical_crossentropy", metrics=["accuracy"])
    return model
```

```
def build_bilstm_model(seq_len: int, n_features: int, n_actions: int):
    _require_tensorflow()
    from tensorflow.keras.layers import Bidirectional, Dense, Dropout, Input, LSTM
    from tensorflow.keras.models import Sequential

    model = Sequential(
        [
            Input(shape=(seq_len, n_features)),
            Bidirectional(LSTM(128, return_sequences=True)),
            Dropout(0.3),
            Bidirectional(LSTM(64)),
            Dropout(0.3),
            Dense(32, activation="relu"),
            Dense(n_actions, activation="softmax"),
        ]
    )
    model.compile(optimizer="adam", loss="categorical_crossentropy", metrics=["accuracy"])
    return model
```

3.3.2. Code train và đánh giá 3 mô hình

```

builders = {
    "RNN": build_rnn_model,
    "LSTM": build_lstm_model,
    "BiLSTM": build_bilstm_model,
}

output = Path(output_dir)
output.mkdir(parents=True, exist_ok=True)

metrics: Dict[str, Dict[str, float]] = {}
histories: Dict[str, Dict[str, list]] = {}

for name, builder in builders.items():
    print(f"\nTraining {name}...")
    model = builder(seq_len, prepared.n_features, prepared.n_actions)
    model.compile(
        optimizer=Adam(learning_rate=learning_rate),
        loss=CategoricalCrossentropy(label_smoothing=0.05),
        metrics=["accuracy"],
    )

    callbacks = [
        EarlyStopping(monitor="val_loss", patience=8, restore_best_weights=True),
        ReduceLROnPlateau(monitor="val_loss", factor=0.5, patience=3, min_lr=1e-5, verbose=1),
    ]

```

```

history = model.fit(
    prepared.X_train,
    y_train_oh,
    epochs=epochs,
    batch_size=batch_size,
    validation_split=0.2,
    verbose=1,
    callbacks=callbacks,
    class_weight=class_weight_map,
)

probs = model.predict(prepared.X_test, verbose=0)
y_pred = np.argmax(probs, axis=1)

metrics[name] = {
    "accuracy": float(accuracy_score(prepared.y_test, y_pred)),
    "precision": float(precision_score(prepared.y_test, y_pred, average="weighted", zero_division=0)),
    "recall": float(recall_score(prepared.y_test, y_pred, average="weighted", zero_division=0)),
    "f1": float(f1_score(prepared.y_test, y_pred, average="weighted", zero_division=0)),
}

```

3.3.3. Code chọn model_best theo F1


```

def select_best_model(artifacts_dir: str) -> str:
    artifacts = Path(artifacts_dir)
    metrics_path = artifacts / "metrics.json"

    if not metrics_path.exists():
        raise FileNotFoundError(f"Missing file: {metrics_path}")

    with metrics_path.open("r", encoding="utf-8") as f:
        metrics = json.load(f)

    best_name = max(metrics.keys(), key=lambda name: metrics[name]["f1"])
    best_model_src = artifacts / f"model_{best_name.lower()}.keras"
    best_model_dst = artifacts / "model_best.keras"

    if not best_model_src.exists():
        raise FileNotFoundError(f"Model file not found: {best_model_src}")

    shutil.copy2(best_model_src, best_model_dst)

    print(f"Best model: {best_name}")
    print(f"F1 score: {metrics[best_name]['f1']:.4f}")
    print(f"Accuracy: {metrics[best_name]['accuracy']:.4f}")
    print(f"Saved best model to: {best_model_dst}")

    return best_name

```

3.3.4. Code vẽ biểu đồ so sánh

```

def plot_comparison(metrics: dict, output_dir: Path) -> None:
    metric_names = ["accuracy", "precision", "recall", "f1"]
    model_names = list(metrics.keys())

    fig, axes = plt.subplots(2, 2, figsize=(12, 9))
    colors = [
        "#CC5A71", "#2D87BB", "#6FAF3A"
    ]

    for i, metric in enumerate(metric_names):
        ax = axes[i // 2, i % 2]
        values = [metrics[name][metric] for name in model_names]
        ax.bar(model_names, values, color=colors[: len(model_names)])
        ax.set_title(f"{metric.upper()} Comparison")
        ax.set_ylim(0, 1)
        ax.set_ylabel(metric)

    fig.tight_layout()
    output_path = output_dir / "model_comparison.png"
    fig.savefig(output_path, dpi=300)
    plt.close(fig)
    print(f"Saved comparison chart: {output_path}")

```

```

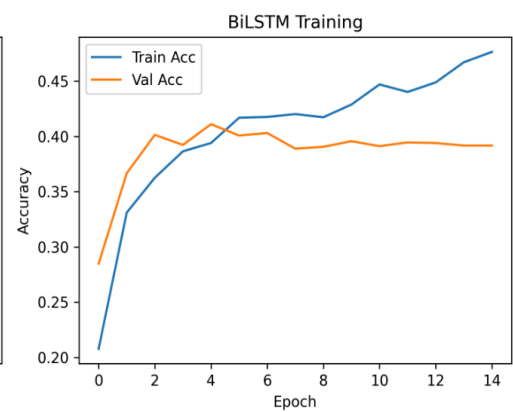
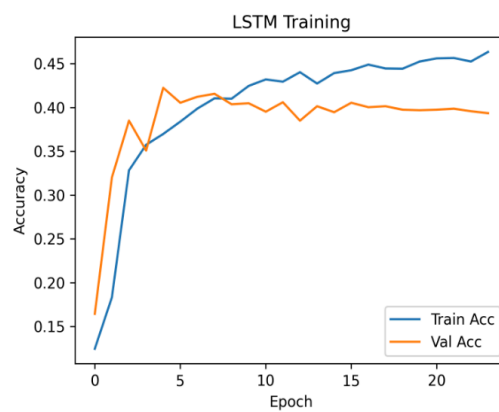
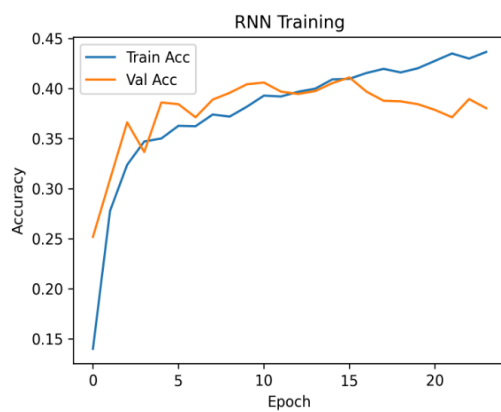
def plot_training_history(history_data: dict, output_dir: Path) -> None:
    model_names = list(history_data.keys())
    fig, axes = plt.subplots(1, len(model_names), figsize=(5 * len(model_names), 4))

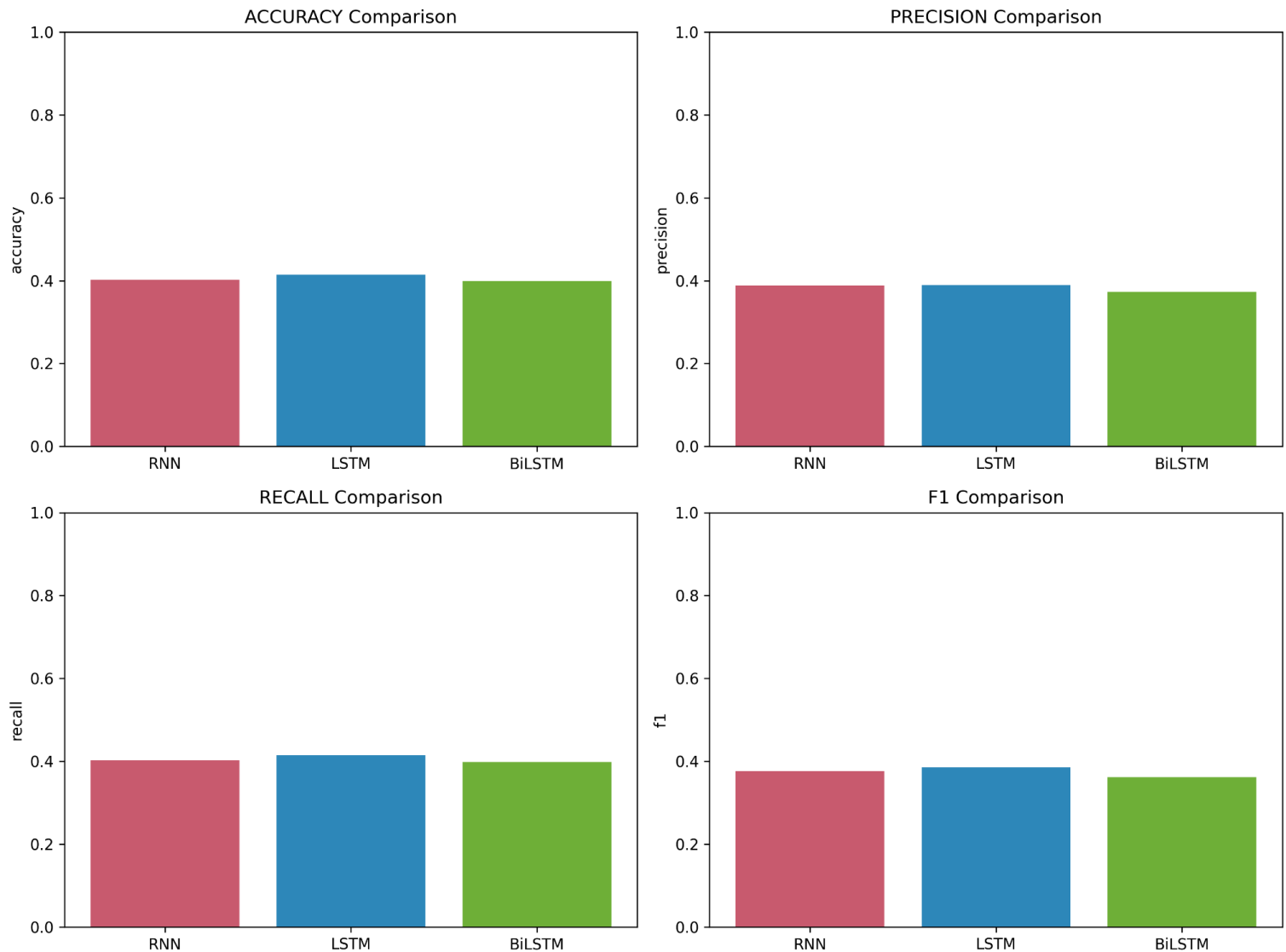
    if len(model_names) == 1:
        axes = [axes]

    for idx, model_name in enumerate(model_names):
        history = history_data[model_name]
        ax = axes[idx]
        ax.plot(history.get("accuracy", []), label="Train Acc")
        ax.plot(history.get("val_accuracy", []), label="Val Acc")
        ax.set_title(f"{model_name} Training")
        ax.set_xlabel("Epoch")
        ax.set_ylabel("Accuracy")
        ax.legend()

    fig.tight_layout()
    output_path = output_dir / "training_history.png"
    fig.savefig(output_path, dpi=300)
    plt.close(fig)
    print(f"Saved training history chart: {output_path}")

```





Nhận xét ngắn gọn cho 2 biểu đồ:

- Biểu đồ so sánh cho thấy LSTM là mô hình tốt nhất trong 3 mô hình, vì Accuracy, Precision, Recall và F1 đều nhỉnh hơn RNN và BiLSTM. RNN đứng thứ hai, còn BiLSTM thấp nhất. Chênh lệch không quá lớn nên có thể nói ba mô hình đều học được tín hiệu từ dữ liệu, nhưng LSTM cho kết quả ổn định hơn. Bạn có thể chèn biểu đồ này tại `model_comparison.png`.
- Biểu đồ training history cho thấy cả 3 mô hình đều tăng nhanh ở vài epoch đầu, sau đó validation accuracy chững lại trong khi train accuracy vẫn tăng. Điều này cho thấy mô hình bắt đầu học khớp với tập train nhiều hơn, tức là có dấu hiệu overfitting nhẹ. LSTM vẫn giữ khoảng cách train-val hợp lý hơn hai mô hình còn lại nên đáng chọn làm `model_best`. Bạn có thể chèn biểu đồ này tại `training_history.png`.

3.4. Kết quả đánh giá thực nghiệm

Kết quả:

```

1  {
2      "RNN": {
3          "accuracy": 0.4024556616643929,
4          "precision": 0.3884416812539439,
5          "recall": 0.4024556616643929,
6          "f1": 0.37663515564073924
7      },
8      "LSTM": {
9          "accuracy": 0.4147339699863574,
10         "precision": 0.3893014338445971,
11         "recall": 0.4147339699863574,
12         "f1": 0.3848260941195951
13     },
14     "BiLSTM": {
15         "accuracy": 0.39836289222373805,
16         "precision": 0.37292440159567025,
17         "recall": 0.39836289222373805,
18         "f1": 0.3613392502605389
19     }
20 }

```

Kết luận chọn model:

1. Mô hình tốt nhất là LSTM vì có F1 cao nhất.
2. File model tốt nhất đã lưu: model_best.keras.

3.5. Lý giải vì sao chọn LSTM

1. Bài toán là dự đoán hành vi theo chuỗi thời gian tương tác user.
2. LSTM giữ ngữ cảnh tốt hơn RNN cơ bản nên ổn định hơn với chuỗi dài.
3. BiLSTM trong bài này không vượt LSTM do đặc tính dữ liệu hành vi theo thời gian một chiều.
4. Với mục tiêu tích hợp hệ e-commerce, LSTM đạt cân bằng tốt giữa chất lượng và chi phí suy luận.

4. Xây dựng KnowledgeBase Graph với Neo4j

4.1. Mục tiêu

Ở câu 2b, em xây dựng Knowledge Graph từ tập dữ liệu hành vi người dùng để biểu diễn quan hệ giữa:

1. user
2. product
3. action
4. category

Mục tiêu của graph là cho phép truy vấn hành vi và quan hệ sản phẩm một cách có cấu trúc, phục vụ cho các bước RAG và recommendation ở các câu sau.

4.2. Dữ liệu đầu vào và quy mô graph

Kết quả graph đã sinh trong ai-service/knowledge_graph_output/graph_summary.json cho thấy:

1. 500 users
2. 100 products
3. 10 categories
4. 14991 events

Các category xuất hiện nhiều nhất:

1. ulcer_hp_support
2. elderly_weak_digestion
3. consultation_package
4. reflux_heartburn
5. test_kit_home

4.3. Ý nghĩa của KB_Graph

Knowledge Graph giúp mô hình hóa hành vi mua sắm y tế theo dạng quan hệ:

1. user thực hiện hành vi nào
2. hành vi đó gắn với sản phẩm nào
3. sản phẩm thuộc nhóm nào
4. các sản phẩm nào thường đi cùng nhau

Từ đó, hệ thống có thể:

1. truy vết hành trình người dùng
2. phân tích nhóm sản phẩm được quan tâm
3. hỗ trợ truy vấn cho chat/RAG
4. hỗ trợ gợi ý sản phẩm theo ngữ cảnh

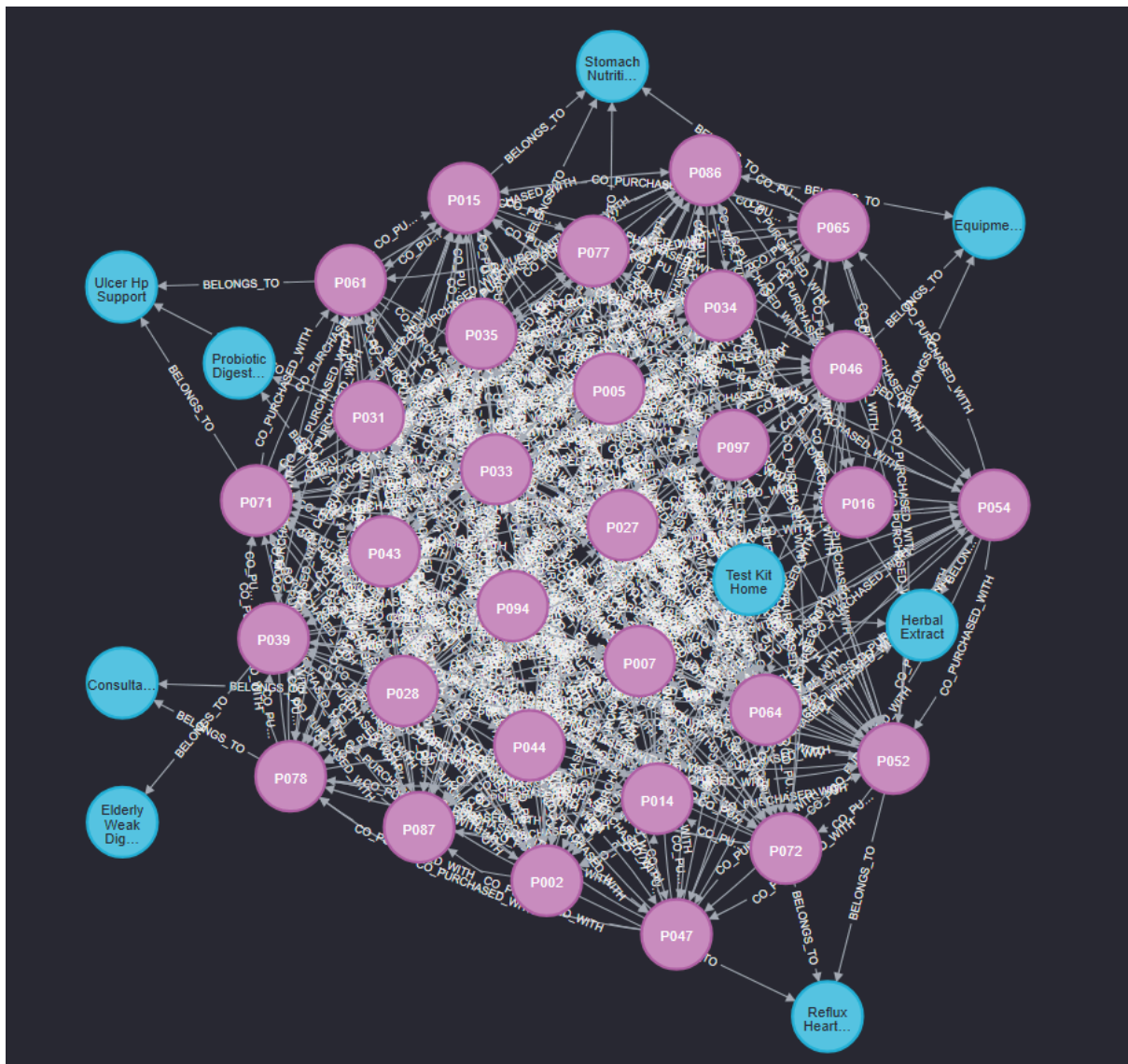
4.4. Quy mô graph

```
1 MATCH (u:User) WITH count(u) AS users
2 MATCH (p:Product) WITH users, count(p) AS products
3 MATCH (c:Category) WITH users, products, count(c) AS categories
4 MATCH ()-[r]->() RETURN users, products, categories, count(r) AS events;
```

	users	products	categories	events
1	500	100	10	54913

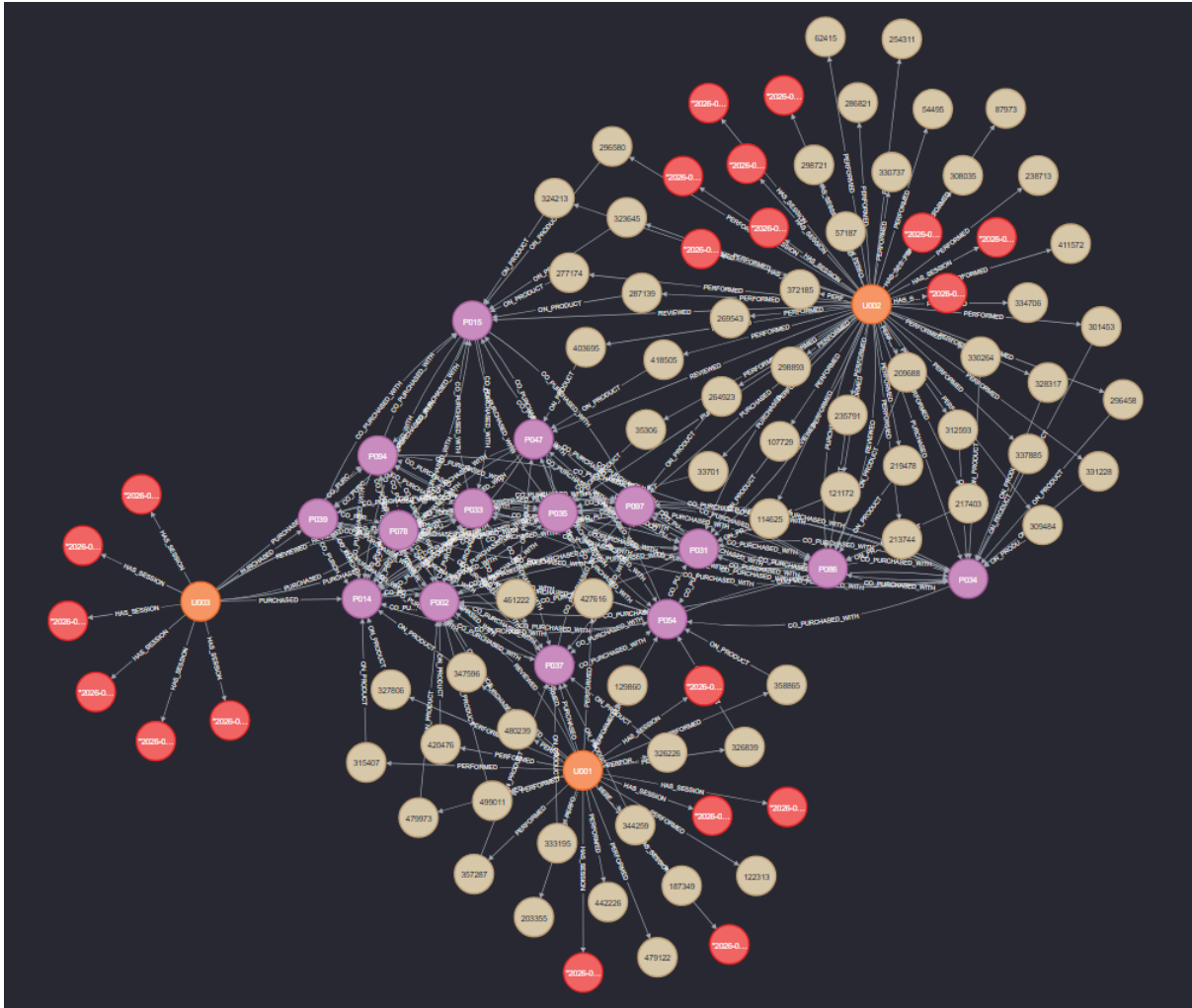
4.4. 20 dòng mẫu dữ liệu graph

id	name	status	category
1	PURCHASED	not	
2	PURCHASED	not	
3	PURCHASED	not	
4	PURCHASED	not	
5	PURCHASED	not	
6	PURCHASED	not	
7	PURCHASED	not	
8	PURCHASED	not	
9	PURCHASED	not	
10	PURCHASED	not	
11	PURCHASED	not	
12	PURCHASED	not	
13	PURCHASED	not	
14	PURCHASED	not	
15	PURCHASED	not	
16	REVIEWED	not	
17	REVIEWED	not	
18	REVIEWED	not	
19	REVIEWED	not	
20	REVIEWED	not	
21	REVIEWED	not	
22	REVIEWED	not	



4.5. Ảnh chi tiết quan hệ người dùng và đồng mua

Ảnh này thể hiện rõ cách người dùng đi từ xem sản phẩm, click, thêm giỏ, mua hàng đến các quan hệ sản phẩm thường đi cùng nhau.



4.6. Kết luận

KB_Graph bằng Neo4j đã được xây dựng thành công từ tập dữ liệu data_user500.csv và đáp ứng đúng yêu cầu câu 2b. Qua các ảnh và bảng truy vấn, có thể thấy dữ liệu đã được tổ chức theo cấu trúc quan hệ rõ ràng giữa User, Product, Category và hành vi tương tác. Hệ graph không chỉ biểu diễn được hành trình người dùng mà còn thể hiện được các quan hệ đồng mua giữa sản phẩm, từ đó hỗ trợ tốt cho phân tích hành vi và gợi ý sản phẩm. Các minh chứng gồm ảnh graph tổng quan, bảng 20 dòng User-Action-Product và bảng 20 dòng co-purchase cho thấy KB_Graph có đủ độ phức tạp và tính ứng dụng thực tế. Đây là nền tảng trực tiếp để triển khai phần 2c (RAG + chat) và 2d (tích hợp vào hệ e-commerce) ở các phần tiếp theo.

5. Tính năng RAG Chat và Tích hợp E-commerce

5.1. Mục tiêu

Ở phần này, em triển khai:

1. Câu 2c: RAG + Chat dựa trên KB_Graph và tri thức đã xây dựng.
2. Câu 2d: Tích hợp AI vào hệ e-commerce để hiển thị gợi ý sản phẩm và giao diện chat cho người dùng cuối.

5.2. Câu 2c: RAG và Chat dựa trên KB_Graph

Kiến trúc xử lý câu hỏi của AI service:

1. Nhận câu hỏi từ người dùng.
2. Phân loại intent.
3. Truy vấn ngữ cảnh từ graph và KB chunks.
4. Tổng hợp ngữ cảnh và sinh câu trả lời.
5. Trả về response dạng JSON cho gateway/UI.

Thành phần triển khai chính:

1. API runtime và endpoint chat trong ai-service/main.py
2. Engine RAG trong ai-service/graph_rag/rag_engine.py
3. Loader KB trong ai-service/kb/kb_loader.py
4. Seed/runner graph trong ai-service/knowledge_graph/run_knowledge_graph.py

5.3. Minh chứng câu 2c

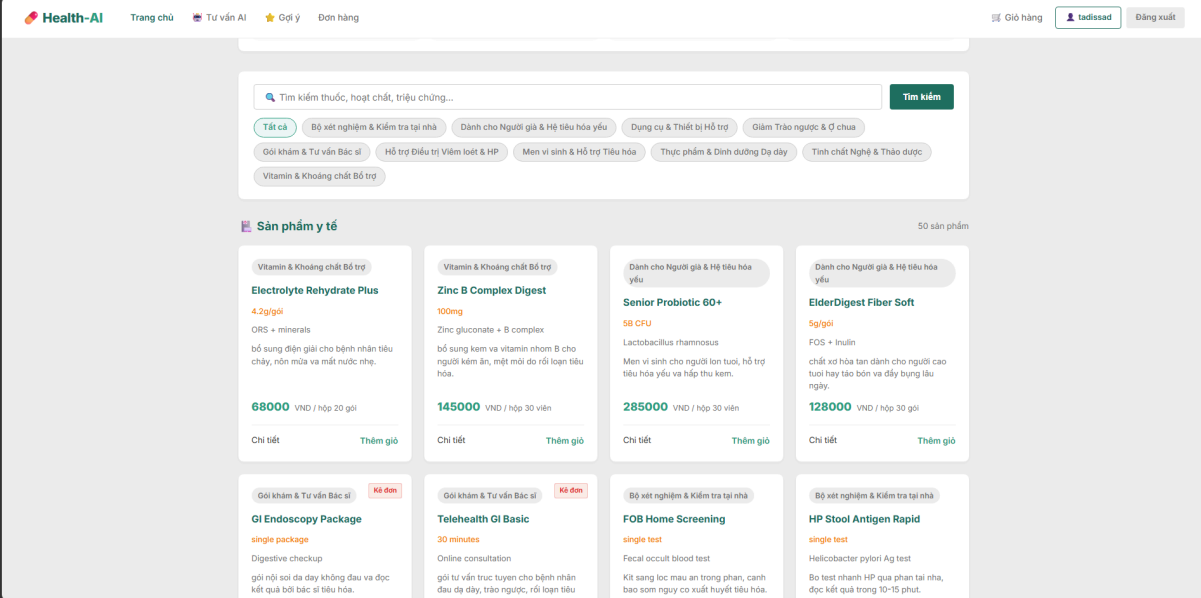
1. Kiểm tra health AI service

Invoke-RestMethod -Method Get -Uri "<http://localhost:8010/health/>"

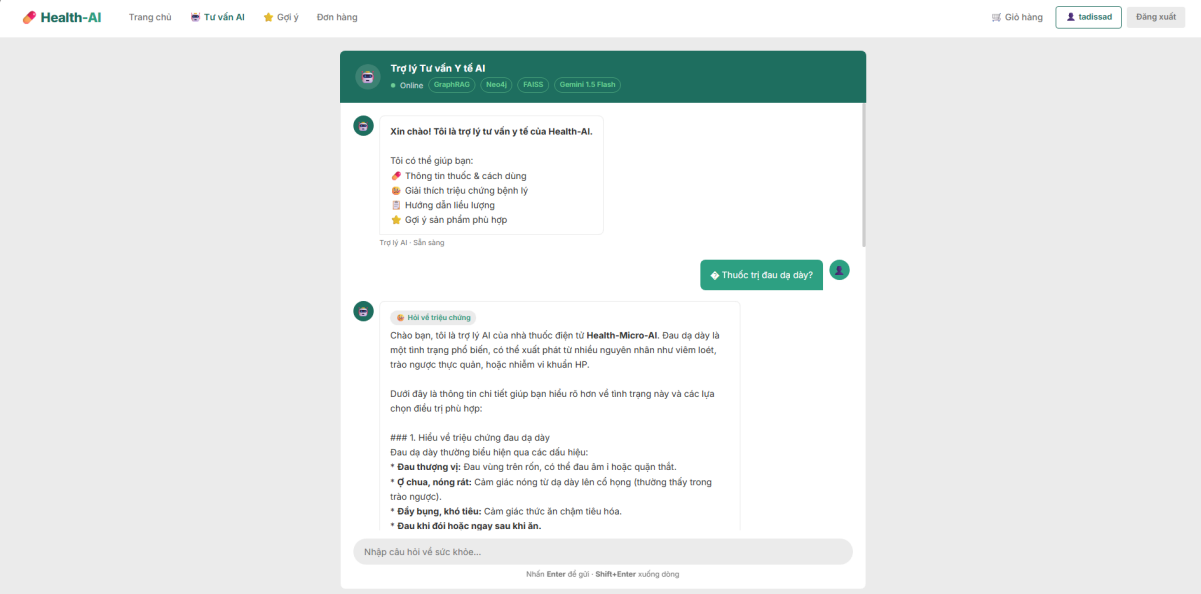
```
PS C:\kientructhietkephanmem\health-micro-ai> Invoke-RestMethod -Method Get -Uri
"http://localhost:8010/health/" | ConvertTo-Json -Depth 6
{
  "status": "UP",
  "service": "ai-service",
  "kb_initialized": true,
  "faiss_chunks": 100,
  "faiss_index": "ready",
  "neo4j": "available"
}
```

2. Kiểm tra trạng thái KB

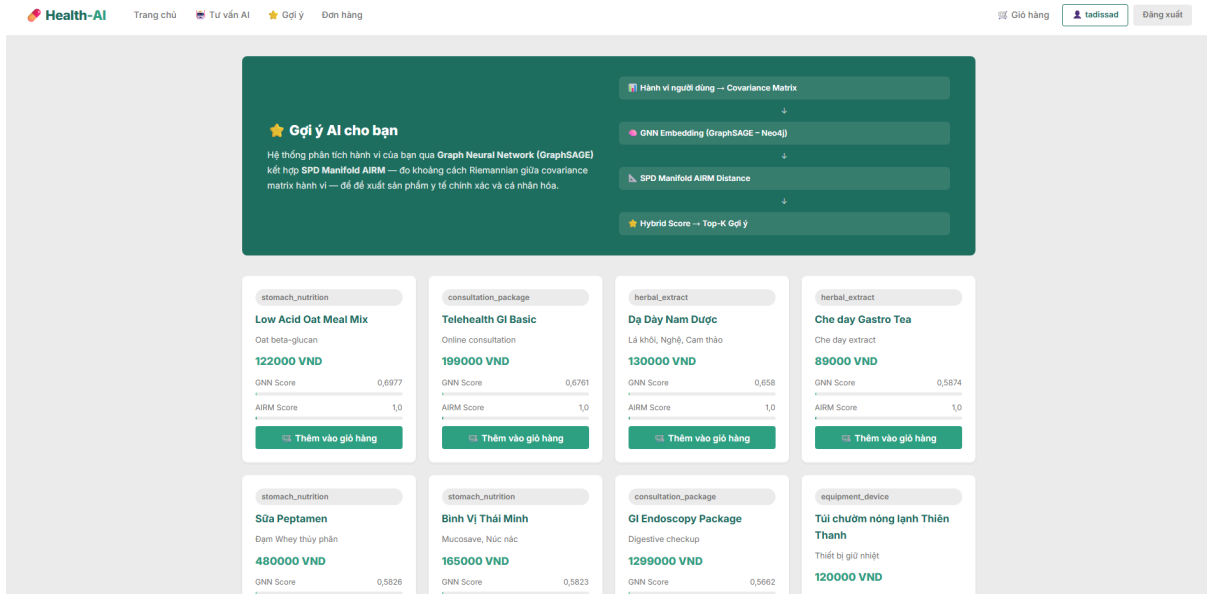
Invoke-RestMethod -Method Get -Uri "http://localhost:8010/api/kb/stats"



2. Giao diện chat riêng trong hệ thống, không dùng giao diện ChatGPT.



3. Giao diện recommend riêng cho người dùng đã đăng nhập.



Điểm tích hợp chính:

1. Gateway view chuyển tiếp chat sang AI trong `api-gateway/app/views.py`
2. Route UI chat và recommend trong `api-gateway/api_gateway/urls.py`
3. Giao diện web:
 - `api-gateway/templates/index.html`
 - `api-gateway/templates/ai_chat.html`
 - `api-gateway/templates/ai_recommend.html`

5.5. Kết luận

Phần 2c và 2d đã được triển khai đầy đủ theo hướng ứng dụng thực tế. Ở câu 2c, hệ thống AI đã xây dựng được luồng RAG + Chat dựa trên tri thức từ KB và KB_Graph, thể hiện qua các endpoint hoạt động ổn định như health, kb stats, chat và graph query. Ở câu 2d, AI đã được tích hợp trực tiếp vào hệ e-commerce thông qua API Gateway, bao gồm giao diện danh sách sản phẩm, màn hình chat nội bộ và màn hình gợi ý sản phẩm cho người dùng. Như vậy, toàn bộ pipeline từ dữ liệu hành vi -> tri thức graph -> AI suy luận -> hiển thị trên giao diện người dùng đã thông suốt, đáp ứng đúng yêu cầu bài tập và đủ cơ sở minh chứng bằng lệnh chạy, kết quả API và ảnh chụp hệ thống.